TEC | Tecnológico de Costa Rica

Escuela de Ingeniería en Computación
Ingeniería en Computación
IC6600 - Principios de Sistemas Operativos

I Proyecto Programado

GutenPress: compresión y descompresión de Huffman con procesos e hilos

Valerie M. Hernández Fernández
valeriehernandez@estudiantec.cr
2020010829

San José, Costa Rica
Septiembre 2026

Índice general

1. Introducción **2**
- 1.1. Reseña histórica del algoritmo de Huffman . . . 2
- 1.2. Explicación del algoritmo de Huffman . . . 3
- 1.3. Paralelismo y concurrencia . . . 4

2. Desarrollo **5**
- 2.1. Instrucciones de instalación y código fuente . . . 5
- 2.2. Implementación del algoritmo de Huffman . . . 12
  - 2.2.1. Organización del código . . . 12
  - 2.2.2. Formato del archivo comprimido . . . 13
  - 2.2.3. Compresión de un archivo . . . 14
  - 2.2.4. Códigos canónicos . . . 14
  - 2.2.5. Descompresión y verificación . . . 15
  - 2.2.6. Versión serial . . . 16
- 2.3. Solución paralela con `fork()` e IPC . . . 16
  - 2.3.1. Reparto del trabajo . . . 16
  - 2.3.2. Estrategia de IPC: tuberías anónimas . . . 16
  - 2.3.3. Sincronización de los subprocesos . . . 17
  - 2.3.4. Descompresor paralelo . . . 17
- 2.4. Solución concurrente con `pthread()` y memoria compartida . . . 18
  - 2.4.1. Memoria compartida . . . 18
  - 2.4.2. Sincronización de los hilos . . . 18
- 2.5. Interfaz gráfica y medición de tiempos . . . 19
- 2.6. Resultados . . . 20

3. Conclusiones **22**

Referencias **24**

Capítulo 1

Introducción

Este informe documenta GutenPress, un compresor y descompresor de directorios escrito en C que emplea el algoritmo de Huffman y verifica la integridad de cada archivo mediante firmas MD5. El mismo algoritmo se implementó en tres versiones. La primera es serial; la segunda es paralela y se basa en subprocesos creados con `fork()` que se comunican por IPC; la tercera es concurrente y utiliza hilos POSIX que comparten memoria. Adicionalmente, una interfaz gráfica desarrollada en GTK ejecuta las tres versiones sobre el mismo directorio y presenta sus resultados en una tabla comparativa.

El conjunto de prueba está formado por los 100 libros más descargados de los últimos 30 días en Project Gutenberg, en formato de texto plano (Project Gutenberg, s.f.).

1.1. Reseña histórica del algoritmo de Huffman

La base teórica de la compresión sin pérdida proviene de la teoría de la información de Claude Shannon. En 1948, Shannon formalizó la medida de información de una fuente y demostró que existe un límite inferior para la cantidad promedio de bits necesaria para representar sus símbolos, conocido como entropía (Shannon, 1948). A partir de ese trabajo surgieron métodos de codificación de longitud variable que asignaban códigos más cortos a los símbolos más frecuentes. Entre ellos destacó el método de Shannon y Robert Fano, el cual no garantizaba un resultado óptimo.

En 1951, David A. Huffman cursaba estudios de posgrado en el Instituto Tecnológico de Massachusetts (MIT) y asistía al curso de teoría de la información impartido por Fano. El profesor ofreció a sus estudiantes la opción de presentar un trabajo escrito en lugar del examen final; el problema propuesto consistía en encontrar el código binario más eficiente. Tras varios meses sin avances, Huffman halló la solución al invertir el enfoque del método de Shannon–Fano: en vez de dividir el alfabeto desde arriba, construyó el árbol de codificación desde abajo, a partir de los símbolos menos frecuentes (Stix, 1991). Posteriormente, en 1952

publicó el resultado como un método para construir códigos de redundancia mínima (Huffman, 1952).

Más de cuatro décadas después, la codificación de Huffman seguía vigente en redes de computadoras, módems y televisión de alta definición (Stix, 1991). Por otra parte, una variante de especial importancia práctica es el código canónico, descrito por Schwartz y Kallick (1964) en el cual los códigos se derivan únicamente de sus longitudes. Gracias a esta propiedad, el código puede almacenarse de forma muy compacta dentro de un archivo comprimido; por esta razón GutenPress utiliza dicha variante.

1.2. Explicación del algoritmo de Huffman

El algoritmo de Huffman produce un *código prefijo*, es decir, un código en el que ninguna palabra es prefijo de otra. En consecuencia, una secuencia de bits puede decodificarse de izquierda a derecha sin ambigüedad y sin separadores entre símbolos. Además, entre todos los códigos prefijo posibles para un conjunto de frecuencias dado, el de Huffman minimiza la longitud promedio de la codificación (Huffman, 1952).

Se trata de un algoritmo voraz que se describe en tres pasos (Cormen y cols., 2022):

1. Se cuenta la frecuencia de cada símbolo y se crea un nodo hoja por símbolo, cuyo peso es su frecuencia. A continuación, todos los nodos se insertan en una cola de prioridad ordenada por peso.

2. Mientras quede más de un nodo en la cola, se extraen los dos de menor peso y se unen como hijos de un nodo nuevo. El peso de este nodo es la suma de los pesos de sus hijos y el nodo se inserta de nuevo en la cola.

3. El nodo restante es la raíz del árbol. Finalmente, el código de cada símbolo corresponde al camino desde la raíz hasta su hoja: cada desplazamiento a la izquierda aporta un 0 y cada desplazamiento a la derecha, un 1.

Dado que los símbolos poco frecuentes se unen primero, estos quedan en los niveles más profundos del árbol y reciben los códigos más largos. Por el contrario, los símbolos frecuentes permanecen cerca de la raíz y obtienen códigos cortos. Con n símbolos distintos y una cola de prioridad implementada como montículo mínimo, la construcción del árbol tiene un costo de $O(n \log n)$ (Cormen y cols., 2022).

El cuadro 1.1 ilustra el procedimiento con el texto `AAAAABBCD`. En primer lugar, los nodos `C` y `D`, ambos de peso 1 se unen en un nodo de peso 2. Luego, este nodo se une con `B`, también de peso 2, en un nodo de peso 4. Por último, el nodo de peso 4 se une con `A`, de peso 5, para formar la raíz. Con códigos de longitud fija se requerirían 2 bits por símbolo, es decir, 18 bits en total. En cambio, el código resultante emplea $5 \cdot 1 + 2 \cdot 2 + 1 \cdot 3 + 1 \cdot 3 = 15$ bits.

Símbolo	Frecuencia	Longitud	Código canónico
A	5	1	`0`
B	2	2	`10`
C	1	3	`110`
D	1	3	`111`

Cuadro 1.1: Código de Huffman para el texto `AAAAABBCD` (ejemplo propio)

1.3. Paralelismo y concurrencia

Aunque con frecuencia se usan como sinónimos, ambos términos describen propiedades distintas de un sistema. Un sistema es *concurrente* cuando permite que varias tareas avancen durante un mismo intervalo de tiempo, aunque no necesariamente en el mismo instante. Por ejemplo, en un procesador de un solo núcleo, el planificador alterna entre las tareas y crea la apariencia de simultaneidad. En cambio, un sistema es *paralelo* cuando puede ejecutar más de una tarea exactamente al mismo tiempo, lo cual requiere varios núcleos o procesadores. Por consiguiente, puede existir concurrencia sin paralelismo; sin embargo, el paralelismo implica tareas concurrentes que se ejecutan en unidades de procesamiento distintas (Silberschatz y cols., 2018).

La misma obra distingue dos formas de dividir el trabajo. En el *paralelismo de datos*, un mismo cálculo se aplica a subconjuntos distintos de los datos; en el *paralelismo de tareas*, en cambio, se distribuyen operaciones diferentes (Silberschatz y cols., 2018). GutenPress aplica paralelismo de datos ya que cada trabajador ejecuta el mismo algoritmo de compresión sobre un subconjunto distinto de los archivos del directorio.

En un sistema operativo tipo UNIX, estas ideas se materializan mediante dos mecanismos. Por un lado, los *subprocesos* creados con `fork()` poseen espacios de memoria separados; por ello, deben comunicarse de forma explícita mediante algún mecanismo de comunicación entre procesos (IPC) (Kerrisk, 2010). Por otro lado, los *hilos* de un mismo proceso comparten la memoria global y el montículo. De este modo, se comunican directamente a través de variables compartidas, aunque requieren sincronización para evitar condiciones de carrera (The Linux man-pages project, s.f.-g). Las dos versiones paralelas de este proyecto corresponden respectivamente a cada uno de estos mecanismos.

Capítulo 2

Desarrollo

2.1. Instrucciones de instalación y código fuente

El código fuente está disponible en el siguiente repositorio de GitHub:

`https://github.com/valeriehernandez-7/GutenPress`

Las instrucciones de esta sección se validaron en Debian 13 con el entorno de escritorio GNOME instalado con las opciones por defecto. Para seguirlas se requiere una conexión a Internet y la contraseña de administrador del equipo. Los pasos deben realizarse en el orden indicado. Cada uno explica su propósito, el comando que debe escribirse y el resultado esperado.

Paso 1. Abrir una terminal. Todos los comandos de esta sección se escriben en una terminal, que es la aplicación donde se ingresan instrucciones de texto. Para abrirla, haga clic en **Actividades**, en la esquina superior izquierda de la pantalla, o presione la tecla con el logotipo de Windows. Luego escriba `terminal` en el buscador y presione **Enter**. Según la versión de GNOME, la aplicación puede figurar con el nombre **Terminal** o **Consola**; cualquiera de las dos es válida.

Al abrirse, la ventana muestra una línea de texto que termina en el símbolo `$`. Esta línea indica que la terminal está lista para recibir un comando. Para ejecutar cada comando de esta guía, escríbalo exactamente como aparece o cópielo y péguelo con **Ctrl+Shift+V**. A continuación, presione **Enter** y espere a que vuelva a aparecer la línea terminada en `$` antes de escribir el siguiente comando.

Paso 2. Instalar las herramientas necesarias. El programa se distribuye como código fuente. Por lo tanto, antes de utilizarlo es necesario instalar las herramientas que lo descargan y lo convierten en programas ejecutables. Los paquetes requeridos son los siguientes:

- `git`: descarga el código fuente desde GitHub.

- `build-essential`: incluye el compilador de C (`gcc`) y la herramienta `make`, que automatiza la compilación.
- `pkg-config`: indica al compilador la ubicación de las bibliotecas instaladas.
- `libgtk-3-dev`: biblioteca GTK 3, necesaria para compilar la interfaz gráfica.
- `libssl-dev`: biblioteca OpenSSL, empleada para calcular las firmas MD5.

Escriba los dos comandos siguientes, uno a la vez. El primero actualiza la lista de paquetes disponibles; el segundo instala los paquetes anteriores:

```
sudo apt update
sudo apt install -y git build-essential pkg-config libgtk-3-dev libssl-dev
```

La palabra `sudo` ejecuta el comando con permisos de administrador. Por esta razón, la terminal solicita la contraseña del usuario. Durante la escritura, la terminal no muestra caracteres ni asteriscos; este comportamiento es normal. Escriba la contraseña completa y presione **Enter**. La instalación puede tardar varios minutos y muestra numerosas líneas de texto. El proceso concluye cuando vuelve a aparecer la línea terminada en `$`.

Es posible que la terminal responda con un mensaje que indica que el usuario no se encuentra en el archivo `sudoers` (en inglés, `is not in the sudoers file`). Este mensaje significa que durante la instalación de Debian se definió una contraseña para el usuario administrador `root`; en ese caso, el usuario normal no tiene permiso para usar `sudo`. Ejecute entonces el siguiente comando, que realiza la misma instalación como usuario `root`. Cuando la terminal muestre `Contraseña:`, escriba la contraseña de `root`, no la del usuario y presione **Enter**:

```
su -c "apt update && apt install -y git build-essential pkg-config libgtk-3-dev
     libssl-dev"
```

Paso 3. Descargar el código fuente. El siguiente comando descarga una copia completa del repositorio, la cual incluye el código y los 100 libros de prueba. La copia se guarda en una carpeta nueva llamada `GutenPress`, dentro de la carpeta personal del usuario:

```
git clone https://github.com/valeriehernandez-7/GutenPress.git
```

La descarga muestra su progreso y finaliza con una línea similar a `Resolving deltas: 100%` o su equivalente en español. Posteriormente, ingrese a la carpeta descargada con el comando `cd`, que cambia la carpeta en la que trabaja la terminal:

```
cd GutenPress
```

Para confirmar la ubicación, ejecute `ls`, comando que lista el contenido de la carpeta actual. El resultado debe incluir, entre otros elementos, `docs`, `Makefile`, `README.md` y `src`. Todos los comandos de los pasos siguientes deben ejecutarse desde esta carpeta. Si la terminal se cierra y se vuelve a abrir, repita el comando `cd GutenPress` antes de continuar.

Paso 4. Compilar el programa. Compilar consiste en traducir el código fuente en C a programas que el sistema puede ejecutar. El siguiente comando compila las tres versiones del compresor y del descompresor (serial, paralela con `fork()` y concurrente con hilos), así como la interfaz gráfica:

```
make
```

La terminal muestra una línea por cada programa compilado y cada una comienza con `gcc`. La compilación finalizó correctamente si vuelve a aparecer la línea terminada en `$` y ninguna línea contiene la palabra `Error`. Este paso solo es necesario la primera vez.

Paso 5. Verificar la instalación (opcional). Aunque este paso no es obligatorio, permite comprobar que las tres versiones funcionan correctamente antes de usar la interfaz. El siguiente comando comprime los 100 libros de prueba con cada versión y los descomprime. Luego compara byte por byte los archivos obtenidos con los originales:

```
make check
```

La prueba puede tardar algunos minutos y muestra una línea por cada archivo procesado. El resultado es satisfactorio cuando la última línea indica `All versions passed the round-trip check.` En cambio, si alguna versión produce un archivo distinto del original, la prueba se detiene y muestra el nombre del archivo que no coincide. Los archivos generados se guardan en la carpeta `test_output`, la cual puede eliminarse con el comando `make distclean`.

Paso 6. Abrir la interfaz gráfica. Para iniciar la interfaz, ejecute el siguiente comando:

```
make run
```

A continuación se abre una ventana titulada **GutenPress**. Mientras esta ventana permanezca abierta, la terminal queda ocupada y no acepta otros comandos. No cierre la terminal, puesto que esa acción cerraría también la interfaz. Para finalizar, cierre la ventana de GutenPress con el botón de cierre de la esquina superior derecha; entonces la terminal vuelve a mostrar la línea terminada en `$`.

La interfaz siempre debe iniciarse con el comando `make run`, desde la carpeta `GutenPress`. Este comando abre la interfaz desde la carpeta `src/gui`, donde se encuentran sus íconos, su hoja de estilos y las rutas hacia los programas de las tres versiones.

Paso 7. Usar la interfaz gráfica. La ventana principal presenta cuatro opciones, tal como se observa en la figura 2.1. Para seleccionar una opción, haga clic sobre ella. Asimismo, para volver a la ventana principal desde cualquier opción, haga clic en el botón con la flecha ubicado a la izquierda del título.

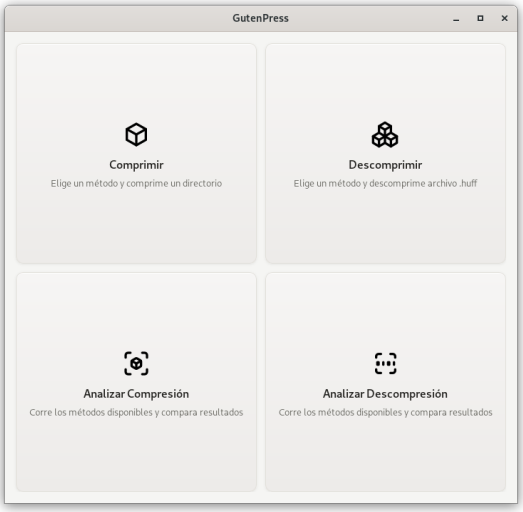


Figura 2.1: Ventana principal de GutenPress

Comprimir. Esta opción comprime una carpeta con una sola versión del programa (figura 2.2). En el campo **Selecciona la carpeta que deseas comprimir**, elija la carpeta que contiene los archivos. Después, seleccione en la lista de métodos la versión que desea utilizar. En el campo **Selecciona dónde guardar el archivo comprimido**, elija la carpeta de destino; luego escriba el nombre del archivo en el campo correspondiente. La extensión `.huff` se agrega automáticamente. Por último, haga clic en **Comprimir**. Al concluir el proceso, la interfaz muestra el tiempo de ejecución y el resumen de la compresión.

Descomprimir. Esta opción expande un archivo `.huff` con una sola versión del programa (figura 2.3). En el campo **Selecciona el archivo .huff a descomprimir**, elija el archivo. A continuación, seleccione la versión en la lista de métodos. En el campo **Selecciona dónde extraer los archivos**, elija la carpeta de destino; en el campo de nombre, escriba el nombre de la carpeta que se creará con los archivos extraídos. Finalmente, haga clic en **Descomprimir**. Al concluir, la interfaz muestra el tiempo de ejecución y la cantidad de firmas MD5 verificadas.

Analizar Compresión. Esta opción comprime la misma carpeta con las tres versiones, una tras otra y presenta una tabla comparativa (figura 2.4). En

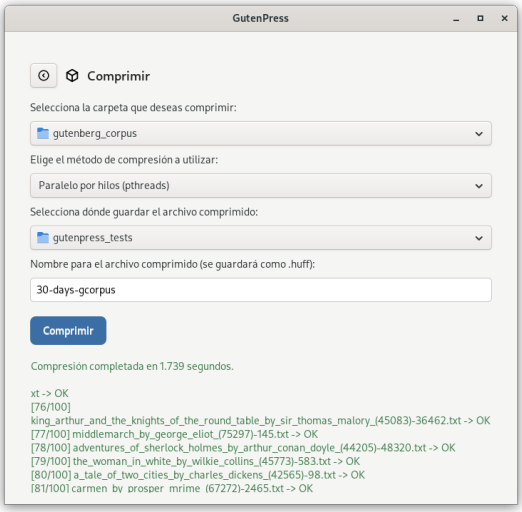


Figura 2.2: Opción **Comprimir** después de comprimir el corpus con la versión con hilos

el campo **Selecciona la carpeta que deseas comprimir**, elija la carpeta. Para utilizar los libros de prueba, abra la carpeta `GutenPress`, luego la carpeta `src` y, dentro de esta, seleccione `gutenberg_corpus`. En el campo **Selecciona la carpeta base donde guardar los resultados**, elija la ubicación de los tres archivos comprimidos. Luego, en el campo **Nombre base para los archivos**, escriba un nombre; el valor predeterminado es `comparacion`. Por último, haga clic en **Ejecutar análisis**.

Al finalizar, la tabla muestra para cada versión el tiempo de ejecución, el porcentaje de aceleración respecto a la versión serial, el tamaño original, el tamaño comprimido, el radio de compresión y el porcentaje de espacio ahorrado. Debajo de la tabla, la interfaz indica cuál fue la versión más rápida.

Analizar Descompresión. Esta opción descomprime el mismo archivo con las tres versiones y presenta una tabla comparativa (figura 2.5). En el campo **Selecciona el archivo .huff a descomprimir**, elija uno de los archivos generados en el análisis de compresión. Cualquiera de los tres es válido, dado que todas las versiones comparten el mismo formato. En el campo **Selecciona la carpeta base donde extraer los resultados**, elija la ubicación de las carpetas de salida; en el campo **Nombre base para las carpetas**, escriba un nombre. Finalmente, haga clic en **Ejecutar análisis**. La tabla resultante muestra para cada versión el tiempo de ejecución, el porcentaje de aceleración, los archivos extraídos, las firmas MD5 verificadas y el porcentaje de salud.

Durante la ejecución de cualquier opción, la interfaz muestra un indicador de carga y desactiva el botón de ejecución. Con los 100 libros de prueba, cada

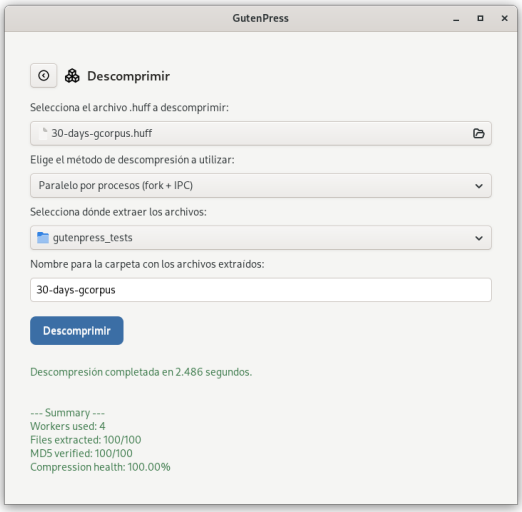


Figura 2.3: Opción **Descomprimir** después de expandir el archivo con la versión con procesos

análisis puede tardar desde unos segundos hasta algunos minutos. Cabe señalar que los tiempos varían entre ejecuciones. Por esta razón, los resultados de rendimiento del proyecto se presentan en la sección 2.6 como el promedio de cinco corridas, de las cuales las figuras 2.4 y 2.5 corresponden a la quinta.

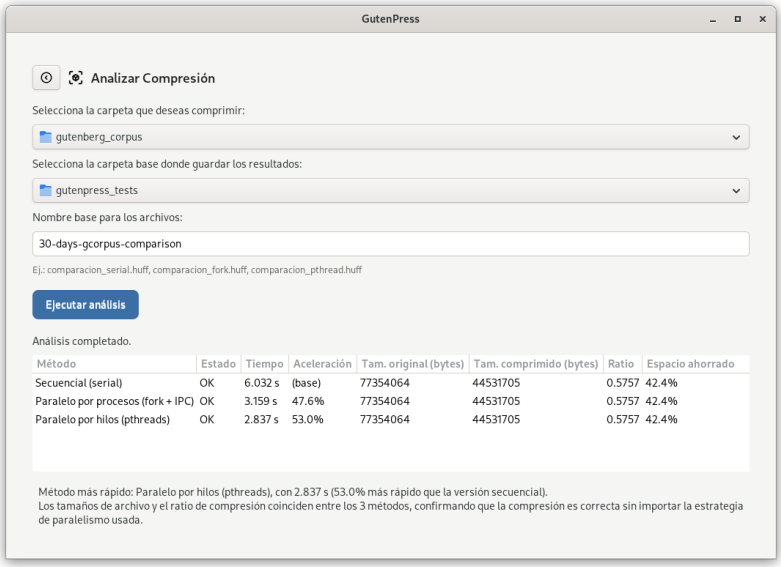


Figura 2.4: Opción **Analizar Compresión** con los resultados de una ejecución sobre el corpus

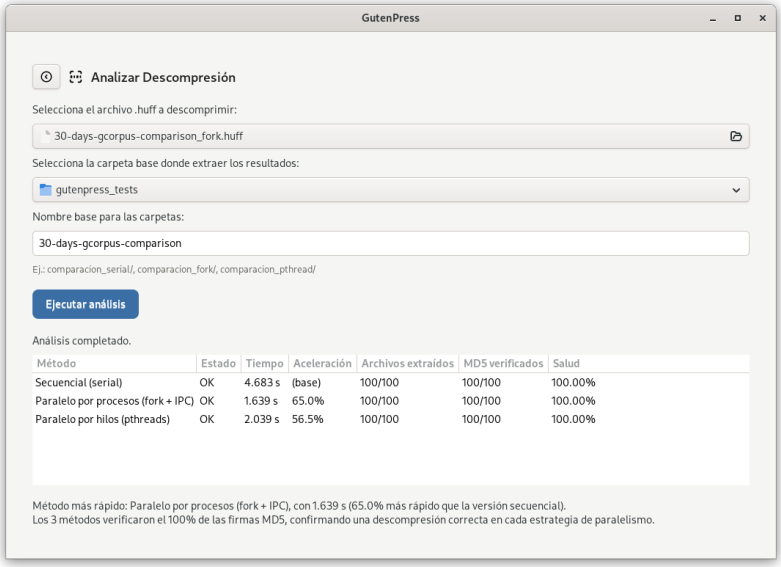


Figura 2.5: Opción **Analizar Descompresión** con los resultados de una ejecución sobre el corpus

Uso por línea de comandos (opcional). Cada versión también puede ejecutarse sin la interfaz, desde la terminal y dentro de la carpeta `GutenPress`. Cada compresor recibe dos argumentos separados por un espacio: la carpeta que se desea comprimir y el nombre del archivo `.huff` que se creará. De forma análoga, cada descompresor recibe el archivo `.huff` y la carpeta donde se extraerán los archivos; si esta carpeta no existe, el programa la crea. Los comandos de las tres versiones son los siguientes:

```
./src/serial/serial_compressor      <carpeta> <archivo.huff>
./src/serial/serial_decompressor    <archivo.huff> <carpeta>
./src/fork_ipc/fork_compressor      <carpeta> <archivo.huff>
./src/fork_ipc/fork_decompressor    <archivo.huff> <carpeta>
./src/pthreads/pthread_compressor   <carpeta> <archivo.huff>
./src/pthreads/pthread_decompressor <archivo.huff> <carpeta>
```

A modo de ejemplo, los siguientes comandos comprimen los libros de prueba con la versión serial en el archivo `libros.huff` y, posteriormente, los extraen en la carpeta `libros_extraidos`:

```
./src/serial/serial_compressor src/gutenberg_corpus libros.huff
./src/serial/serial_decompressor libros.huff libros_extraidos
```

Al finalizar, cada programa imprime un resumen. En el caso del descompresor, el resumen debe indicar `MD5 verified: 100/100` y `Compression health: 100.00%`.

2.2. Implementación del algoritmo de Huffman

2.2.1. Organización del código

Las tres versiones comparten el mismo núcleo, ubicado en `src/common`; así, solo difiere la forma en que cada una reparte el trabajo. Los módulos principales son los siguientes:

- `min_heap`**:** cola de prioridad implementada como montículo mínimo, utilizada para construir el árbol.
- `huffman`**:** conteo de frecuencias, construcción del árbol, cálculo de longitudes, asignación de códigos canónicos, codificación y decodificación.
- `bit_stream`**:** escritura y lectura de bits individuales sobre un búfer de bytes.
- `md5_utils`**:** cálculo de firmas MD5.
- `file_format`**:** estructura, lectura y escritura de las cabeceras del archivo comprimido.
- `compression_pipeline`**:** flujo completo de compresión y descompresión de un archivo individual.

- `file_scanner`, `directory_utils`, `work_partition` y `growable_buffer`: lectura de directorios, manejo de rutas, reparto del trabajo entre trabajadores y búferes de memoria de tamaño variable.

Cada módulo cuenta con una prueba unitaria propia, que se compila con `make tests`. Además, las tres versiones completas se validan con `make check`.

2.2.2. Formato del archivo comprimido

El enunciado solicita expandir *un* archivo comprimido en un directorio. Por esta razón, GutenPress genera un único archivo contenedor `.huff`, similar a un `.zip`, con una entrada por cada archivo original. El contenedor comienza con una cabecera global, a la que siguen las entradas; cada entrada consta de su propia cabecera y de sus datos comprimidos (cuadro 2.1).

Bloque	**Campo**	**Descripción**
Cabecera global	número mágico	Identifica el formato propio (`HUFC`)
	versión	Versión del formato
	cantidad de entradas	Número de archivos contenidos
Cabecera de entrada	nombre	Nombre del archivo original y su longitud
	tamaño original	Bytes del archivo antes de comprimir
	tamaño comprimido	Bytes de datos Huffman que siguen
	firma MD5	16 bytes calculados antes de comprimir
	longitudes de código	256 bytes, uno por cada valor de byte
	bits de relleno	Bits sin uso en el último byte
Datos	flujo de bits	Contenido codificado con Huffman

Cuadro 2.1: Estructura del archivo `.huff`

Las cabeceras se escriben y leen campo por campo, en lugar de volcar las estructuras de C completas con `fwrite()`. Esta decisión obedece a que el compilador puede insertar bytes de relleno entre los campos de una estructura y dicho relleno no está garantizado entre compilaciones distintas. En consecuencia, escribir la estructura completa haría frágil el formato. Durante la lectura, el número mágico se valida en primer lugar; de este modo, un archivo ajeno se rechaza antes de interpretar el resto de su contenido.

El alfabeto de símbolos corresponde a los 256 valores posibles de un byte. Por ello, el compresor funciona con cualquier tipo de archivo y no únicamente

con texto. Asimismo, el campo de firma MD5 satisface el requisito de almacenar la firma entre los metadatos del archivo comprimido.

2.2.3. Compresión de un archivo

La compresión de cada archivo sigue estos pasos:

1. **Firma MD5.** Se lee el archivo completo y se calcula su firma MD5 sobre el contenido original, antes de comprimirlo. MD5 produce un resumen de 128 bits del mensaje (Rivest, 1992). Para este cálculo se utiliza la interfaz EVP de OpenSSL 3, puesto que las funciones MD5 de bajo nivel están obsoletas en esa versión (OpenSSL Project, s.f.).

2. **Frecuencias.** Se recorre el contenido y se cuenta la cantidad de apariciones de cada uno de los 256 valores de byte.

3. **Árbol.** Se construye el árbol de Huffman con el montículo mínimo, según el procedimiento descrito en la sección 1.2.

4. **Longitudes.** Se recorre el árbol para obtener la profundidad de cada hoja, que equivale a la longitud del código de su símbolo. A partir de este punto, el árbol deja de ser necesario.

   Este paso contempla un caso especial. Si el archivo contiene un solo valor de byte distinto, el árbol tendría una única hoja en la raíz y ese símbolo recibiría un código de 0 bits, el cual no puede decodificarse. Para evitarlo, la hoja se une con una hoja ficticia de frecuencia cero, lo que le asigna un código de 1 bit. Por su parte, si el archivo está vacío, no se construye ningún árbol.

5. **Códigos canónicos.** Se asignan los códigos a partir de las longitudes, tal como se explica en la subsección siguiente.

6. **Codificación.** Se recorre nuevamente el contenido y se escribe el código de cada byte en el flujo de bits. Si el último byte queda incompleto, se rellena y se registra la cantidad de bits de relleno.

7. **Escritura.** Se escribe la cabecera de la entrada y, a continuación, el flujo de bits.

2.2.4. Códigos canónicos

Un árbol de Huffman no es único, dado que intercambiar los hijos de cualquier nodo produce otro código igualmente óptimo. El código canónico elimina esta arbitrariedad: conserva la longitud de cada código, pero asigna sus valores numéricos en orden creciente. Así, cada código queda determinado únicamente por su longitud y por el orden de los símbolos (Schwartz y Kallick, 1964).

En GutenPress, los símbolos se ordenan por longitud de código y, en caso de igual longitud, por el valor del byte. Luego se recorren en ese orden con un

contador que inicia en cero. Cada símbolo recibe el valor actual del contador y este se incrementa en uno. Cuando la longitud aumenta, el contador se desplaza hacia la izquierda tantos bits como haya crecido la longitud. El cuadro 1.1 muestra el resultado de este procedimiento para el ejemplo anterior.

La ventaja práctica radica en que el archivo solo necesita almacenar la longitud de cada símbolo, es decir, 256 bytes por entrada, en lugar del árbol completo. A su vez, el descompresor repite el mismo procedimiento y reconstruye exactamente los mismos códigos. Para los 100 libros del corpus, estas tablas suman 25 600 bytes, una fracción despreciable frente al tamaño total.

El ordenamiento de los símbolos se implementó mediante un ordenamiento por conteo sobre las longitudes. Originalmente se empleaba `qsort()` con una función de comparación que accedía al arreglo de longitudes a través de una variable global. No obstante, en la versión con hilos, varios hilos calculaban códigos simultáneamente y sobrescribían esa variable, lo que provocaba una condición de carrera. El ordenamiento por conteo utiliza exclusivamente variables locales, por lo que es seguro entre hilos. Además, resulta más rápido ya que las longitudes solo pueden tomar valores entre 1 y 32. En efecto, basta con contar cuántos símbolos hay de cada longitud y recorrer las longitudes en orden creciente; dentro de cada longitud, los símbolos se recorren según su valor.

2.2.5. Descompresión y verificación

El descompresor lee la cabecera global y procesa cada entrada. Para ello, reconstruye los códigos canónicos a partir de la tabla de longitudes y decodifica el flujo de bits. Luego escribe el archivo en el directorio de destino, calcula su firma MD5 y la compara con la firma almacenada. Al finalizar, informa cuántos archivos se extrajeron y cuántas firmas coincidieron; esta proporción corresponde al porcentaje de salud de la compresión.

La primera versión del decodificador leía un bit a la vez. Después de cada bit, recorría los 256 símbolos en busca de uno cuyo código coincidiera con los bits leídos hasta ese momento. Si bien el resultado era correcto, la medición del tiempo sobre el corpus completo reveló que la descompresión serial tardaba 203.7 segundos.

La solución aprovecha una propiedad del código canónico: todos los códigos de una misma longitud son valores consecutivos. Antes de decodificar, se construye una tabla que almacena, para cada longitud, el primer código, la cantidad de códigos y la posición del primer símbolo en la lista ordenada. Durante la decodificación, los bits leídos se acumulan en un entero. Para la longitud actual, basta con comparar ese valor con el primer código y con la cantidad de códigos de dicha longitud; si el valor se encuentra dentro del rango, el símbolo se obtiene directamente por índice. Esta técnica de decodificación, basada en la estructura canónica, corresponde a la descrita por Moffat y Turpin (1997). Como resultado, el costo por bit pasó de un recorrido de 256 símbolos a una comparación de tiempo constante. De esta manera, la descompresión del corpus se redujo de 203.7 a 3.86 segundos, es decir, fue aproximadamente 53 veces más rápida y conservó las 100 firmas verificadas.

2.2.6. Versión serial

La versión serial (`src/serial`) lista los archivos regulares del directorio de entrada y los procesa uno por uno con el flujo descrito. Cada entrada se escribe directamente en el archivo `.huff`. Por su parte, el descompresor serial recorre las entradas en orden. Esta versión constituye la línea base para calcular la aceleración de las otras dos.

2.3. Solución paralela con `fork()` e IPC

2.3.1. Reparto del trabajo

El compresor (`src/fork_ipc`) consulta el número de núcleos en línea mediante `sysconf(_SC_NPROCESSORS_ONLN)` (The Linux man-pages project, s.f.-h). Con base en ese valor, crea la misma cantidad de trabajadores, sin superar el número de archivos. La lista de archivos se divide en bloques contiguos, uno por trabajador. Cada bloque recibe el cociente entero entre el total de archivos y la cantidad de trabajadores; el residuo, a su vez, se reparte de a un archivo entre los primeros trabajadores. Por consiguiente, los bloques difieren como máximo en un archivo: con 100 archivos y 4 trabajadores, cada uno recibe 25.

Cada trabajador se crea con `fork()`, llamada que produce un proceso hijo con una copia del espacio de memoria del padre. En Linux, esta copia se implementa mediante páginas de copia en escritura; por lo tanto, la lista de archivos no se duplica físicamente mientras no se modifique (The Linux man-pages project, s.f.-b).

2.3.2. Estrategia de IPC: tuberías anónimas

Dado que el padre y los hijos no comparten memoria, los datos comprimidos deben regresar al padre mediante un mecanismo de IPC. Para ello se eligieron tuberías anónimas (`pipe()`) ya que la relación entre los procesos es de padre a hijo y el flujo de datos es unidireccional, del hijo hacia el padre (Kerrisk, 2010).

En primer lugar, el padre crea una tubería por trabajador; posteriormente, crea a todos los hijos. Cada hijo hereda los descriptores de todas las tuberías y no solo los de la suya. Por esta razón, al iniciar, cierra todos los extremos que no le corresponden y conserva únicamente el extremo de escritura de su propia tubería. Este paso es indispensable porque el fin de archivo en una tubería solo se produce cuando se cierran todos sus extremos de escritura, en todos los procesos (The Linux man-pages project, s.f.-d). En otras palabras, una copia olvidada en otro hijo impediría que el padre detectara el final de los datos. Por el mismo motivo, el padre cierra todos los extremos de escritura, puesto que nunca escribe en las tuberías.

El hijo envuelve su descriptor de escritura en un `FILE*` mediante `fdopen()`. Para cada archivo asignado, el hijo invoca `compressFileIntoStream()`, que es la misma función que utiliza la versión serial. Esta función escribe la cabecera de la entrada seguida de los datos comprimidos, en el mismo formato del archivo

`.huff`. Gracias a ello, no fue necesario diseñar un protocolo de serialización nuevo, puesto que por la tubería viajan entradas completas del archivo final.

2.3.3. Sincronización de los subprocesos

El principal riesgo de este diseño es un bloqueo mutuo. La capacidad de una tubería es limitada, con 65 536 bytes por defecto en Linux. Cuando la tubería se llena, la llamada `write()` del hijo se bloquea hasta que otro proceso lea los datos (The Linux man-pages project, s.f.-d). Cada trabajador produce alrededor de 11 MB de datos comprimidos, cantidad muy superior a esa capacidad. Por lo tanto, si el padre leyera las tuberías en orden y esperara a que terminara la primera antes de atender la segunda, el segundo hijo quedaría bloqueado con su tubería llena. Según el orden de ejecución, el programa podría detenerse indefinidamente.

Para evitar este escenario, el padre crea primero a todos los hijos y, a continuación, atiende todas las tuberías simultáneamente mediante `poll()`. Esta llamada se bloquea hasta que al menos uno de los descriptores tenga datos disponibles (The Linux man-pages project, s.f.-e). Cada vez que `poll()` retorna, el padre lee de las tuberías listas bloques de hasta 64 KB y los agrega al búfer de memoria del trabajador correspondiente. Cuando una lectura devuelve cero bytes, la tubería alcanzó el fin de archivo; en ese momento, el padre la cierra y la excluye de las llamadas siguientes a `poll()`. Los búferes se mantienen separados por trabajador, debido a que una lectura puede devolver una entrada incompleta, cortada en cualquier byte. Si todos los datos se mezclaran en un solo búfer, el resultado quedaría corrupto.

En síntesis, la sincronización entre el padre y los hijos se logra mediante tres mecanismos:

- **Bloqueo de la tubería:** regula el ritmo de producción del hijo según el consumo del padre.
- **Fin de archivo:** cuando un hijo termina y cierra su extremo de escritura, el padre recibe el fin de archivo en esa tubería y determina que los datos de ese trabajador están completos (The Linux man-pages project, s.f.-d).
- `waitpid()`**:** el padre espera a cada hijo para liberar sus recursos y revisar su código de salida; así, cualquier fallo en un hijo se detecta y se informa (The Linux man-pages project, s.f.-i).

Una vez que todos los hijos han terminado, el padre escribe la cabecera global y concatena los búferes en el orden de los trabajadores, no en el orden de llegada. De este modo, el archivo resultante es siempre idéntico, independientemente de cuál hijo haya terminado primero.

2.3.4. Descompresor paralelo

Antes de crear a los trabajadores, el padre recorre una sola vez las cabeceras del archivo `.huff`. Con esa información construye un índice que contiene los

datos de cada entrada y la posición en bytes donde comienzan sus datos comprimidos. Luego reparte las entradas entre los trabajadores con el mismo esquema del compresor. Cada hijo abre el archivo de forma independiente, se desplaza directamente a sus entradas con `fseek()`, las decodifica y escribe cada archivo en el directorio de destino. Posteriormente, calcula la firma MD5 del archivo ya escrito y la compara con la firma almacenada; de esta forma, la verificación comprueba el contenido que efectivamente quedó en el disco. Como cada hijo escribe archivos distintos, no se producen conflictos entre ellos.

En este caso no es necesario devolver el contenido de los archivos. Por cada entrada, el hijo envía al padre una estructura de tamaño fijo con dos valores: si la extracción fue exitosa y si la firma coincidió. El padre emplea el mismo esquema basado en `poll()` para recibir estos resultados. Con ellos suma los totales de archivos extraídos y de firmas verificadas, a partir de los cuales calcula el porcentaje de salud. Puesto que los registros son pequeños y de tamaño fijo, no se requiere un búfer por trabajador.

2.4. Solución concurrente con `pthread()` y memoria compartida

2.4.1. Memoria compartida

La versión con hilos (`src/pthreads`) utiliza el mismo número de trabajadores y el mismo reparto que la versión con `fork()`. Sin embargo, cada trabajador es un hilo POSIX creado con `pthread_create()`. Todos los hilos de un proceso comparten la memoria global y el montículo (The Linux man-pages project, s.f.-g); por ende, la memoria compartida es la del propio proceso. La lista de archivos, los búferes de salida y los contadores de progreso son estructuras que todos los hilos pueden leer y modificar directamente, sin copias ni tuberías.

Cada hilo escribe los datos comprimidos de sus archivos en su propio búfer de memoria. Con el fin de reutilizar sin cambios la función que escribe entradas en un `FILE*`, la salida de cada archivo se abre como flujo mediante `open_memstream()`. Esta función crea un `FILE*` cuyo contenido se almacena en un búfer dinámico que crece automáticamente (The Linux man-pages project, s.f.-c). Por último, el hilo principal espera a todos los hilos con `pthread_join()`. Después escribe la cabecera global y concatena los búferes en el orden de los hilos, al igual que la versión con procesos.

2.4.2. Sincronización de los hilos

El diseño procura minimizar el estado compartido que se modifica. Cada hilo dispone de su propio búfer de salida y de sus propios archivos y ningún otro hilo accede a ellos. En consecuencia, estas estructuras no requieren protección.

El recurso compartido que sí se modifica es el contador de progreso, que registra la cantidad de archivos procesados y se imprime con el formato `[k/100]`. Si dos hilos lo incrementaran simultáneamente sin protección, ambos podrían

leer el mismo valor y uno de los incrementos se perdería. Por esta razón, el contador está protegido por un `pthread_mutex_t`. Un hilo adquiere el mutex con `pthread_mutex_lock()`, incrementa el contador, imprime la línea de progreso y libera el mutex con `pthread_mutex_unlock()`. Mientras un hilo posee el mutex, cualquier otro hilo que intente adquirirlo espera hasta su liberación. Así, el incremento y la impresión se ejecutan como una única operación indivisible (The Linux man-pages project, s.f.-f). Además, la sección crítica se mantuvo lo más breve posible, dado que la compresión propiamente dicha ocurre fuera del mutex.

El descompresor sigue el mismo principio, aunque con una precaución adicional. Un mismo `FILE*` mantiene una única posición de lectura. Por lo tanto, si varios hilos ejecutaran `fseek()` y `fread()` sobre el mismo manejador, uno de ellos podría mover la posición mientras otro lee y los datos se mezclarían. Para evitarlo, cada hilo abre su propio `FILE*` sobre el archivo `.huff`. La única estructura compartida de solo lectura es el índice de entradas, que el hilo principal construye antes de crear a los hilos. Dado que cada hilo escribe archivos distintos en el disco, la escritura no requiere sincronización. Los únicos datos compartidos que se modifican son los contadores de archivos extraídos y de firmas verificadas; estos se actualizan, junto con la línea de progreso, dentro de una sección crítica protegida por un segundo mutex.

Cabe destacar que la sincronización también obligó a revisar el código común. La condición de carrera en la asignación de códigos canónicos, descrita en la sección 2.2, no se manifestaba en la versión serial ni en la versión con procesos, porque cada proceso posee su propia copia de la variable global. El error solo apareció cuando varios hilos compartieron la misma memoria.

2.5. Interfaz gráfica y medición de tiempos

La interfaz (`src/gui`) está desarrollada en GTK 3 y ofrece cuatro opciones: comprimir, descomprimir, analizar la compresión y analizar la descompresión. Las dos últimas ejecutan las tres versiones sobre el mismo directorio o archivo y presentan los resultados en una tabla comparativa, como se describió en la sección 2.1.

Cada versión se ejecuta como un subproceso independiente. Para ello, la interfaz crea un hijo con `fork()`, redirige su salida estándar hacia una tubería y lo reemplaza por el ejecutable correspondiente mediante `execv()`. El tiempo se mide desde la interfaz con `clock_gettime()` y el reloj `CLOCK_MONOTONIC`, el cual no se ve afectado por cambios en la hora del sistema (The Linux man-pages project, s.f.-a). El reloj se consulta antes de crear el subproceso y después de que `waitpid()` confirma su finalización. Por consiguiente, la medición incluye el costo completo de cada programa y las tres versiones se miden exactamente de la misma forma. Por su parte, los tamaños, el radio de compresión y el porcentaje de salud se obtienen del resumen que cada programa imprime al terminar.

El porcentaje de aceleración de cada versión paralela se calcula respecto a

la corrida serial mediante la siguiente expresión:

$$\text{aceleración} = \frac{T_{\text{serial}} - T_{\text{versión}}}{T_{\text{serial}}} \times 100$$

Este valor representa el porcentaje del tiempo serial que la versión logra ahorrar. Por ejemplo, una aceleración del 50 % indica que la versión tardó la mitad del tiempo de la versión serial.

Finalmente, la ejecución de cada subproceso ocurre en un hilo secundario, con el propósito de que la ventana continúe respondiendo durante el análisis. El resultado se entrega al hilo principal de GTK mediante `g_idle_add()`, función que programa una rutina para que se ejecute en el ciclo principal de eventos (The GNOME Project, s.f.).

2.6. Resultados

Las pruebas se realizaron en una máquina virtual con Debian 13 y 4 núcleos. El corpus de 100 libros ocupa 77 354 064 bytes, mientras que el archivo comprimido ocupa 44 531 705 bytes; esto corresponde a un radio de compresión de 0.5757, idéntico en las tres versiones. Asimismo, las tres versiones verificaron correctamente las 100 firmas MD5 y los archivos `.huff` que generan son intercambiables. En otras palabras, cualquier descompresor puede expandir el archivo producido por cualquier compresor.

En los cuadros de esta sección, las versiones se identifican con los mismos nombres que emplea la interfaz gráfica: **Secuencial (serial)**, **Paralelo por procesos (fork + IPC)** y **Paralelo por hilos (pthreads)**. En el texto, estas corresponden a la versión serial, la versión con procesos y la versión con hilos, respectivamente.

Para medir los tiempos, se ejecutaron desde la interfaz gráfica cinco análisis de compresión y cinco análisis de descompresión sobre el corpus. Los primeros cuatro análisis de descompresión emplearon el archivo generado por la versión con hilos en el cuarto análisis de compresión; el quinto, en cambio, utilizó el archivo generado por la versión con procesos en el quinto análisis de compresión. En todos los casos, las tres versiones expandieron el archivo correctamente, lo que confirma en la práctica que el formato es intercambiable. La quinta corrida corresponde, además, a las figuras 2.4 y 2.5. El cuadro 2.2 presenta el promedio de las cinco corridas y el cuadro 2.3 detalla los valores de cada corrida. El porcentaje de aceleración se calculó con la expresión de la sección 2.5 sobre los tiempos promedio. Entre paréntesis se indica, además, el factor de aceleración equivalente, $T_{\text{serial}}/T_{\text{versión}}$.

Las dos versiones paralelas redujeron el tiempo a menos de la mitad del tiempo serial, tanto en la compresión como en la descompresión. Entre ellas, la diferencia promedio es de 52 milisegundos en la compresión, a favor de la versión con hilos y de 74 milisegundos en la descompresión, a favor de la versión con procesos. Estas diferencias son mucho menores que la variación entre corridas de una misma versión. Por ejemplo, el tiempo serial de compresión fue de

Versión	**Salud**	**Tiempo compresión**	**Acel. compresión**	**Tiempo descompresión**	**Acel. descompresión**
Secuencial (serial)	100 %	4.588 s	—	4.554 s	—
Paralelo por procesos (fork + IPC)	100 %	2.079 s	54.7 % (2.21×)	1.808 s	60.3 % (2.52×)
Paralelo por hilos (pthreads)	100 %	2.027 s	55.8 % (2.26×)	1.882 s	58.7 % (2.42×)

Cuadro 2.2: Tiempo promedio de cinco corridas y aceleración respecto a la versión serial

	Compresión (s)			**Descompresión (s)**		
Corrida	**Secuencial (serial)**	**Por procesos (fork + IPC)**	**Por hilos (pthreads)**	**Secuencial (serial)**	**Por procesos (fork + IPC)**	**Por hilos (pthreads)**
1	3.921	**1.701**	1.917	4.403	1.726	**1.580**
2	4.178	1.899	**1.839**	4.363	**1.878**	2.208
3	4.903	1.750	**1.623**	5.105	1.939	**1.814**
4	3.908	**1.887**	1.917	4.214	1.856	**1.771**
5	6.032	3.159	**2.837**	4.683	**1.639**	2.039
Promedio	4.588	2.079	2.027	4.554	1.808	1.882

Cuadro 2.3: Tiempos de cada corrida. En negrita, el menor tiempo entre las versiones paralelas en cada corrida, por separado para la compresión y la descompresión

3.908s en la corrida más rápida y de 6.032 s en la más lenta. Por otra parte, el menor tiempo no correspondió siempre a la misma versión: la versión con hilos lo obtuvo en tres de las cinco corridas de compresión y en tres de las cinco de descompresión, mientras que la versión con procesos lo obtuvo en las dos restantes de cada caso. Con base en estos datos, se concluye que ambas versiones paralelas presentan un rendimiento equivalente en este entorno.

Capítulo 3

Conclusiones

1. **El paralelismo redujo el tiempo a menos de la mitad, aunque lejos del máximo teórico.** Con 4 trabajadores, las versiones paralelas fueron entre 2.2 y 2.5 veces más rápidas que la versión serial, en promedio (cuadro 2.2), mientras que el máximo teórico con 4 núcleos sería de 4 veces. Esta diferencia se explica porque una parte del trabajo continúa siendo serial: listar el directorio o leer el índice del archivo, escribir la cabecera, concatenar los resultados y crear y esperar a los trabajadores. La ley de Amdahl establece que dicha fracción serial limita la aceleración, sin importar cuántos procesadores se agreguen (Amdahl, 1967). Según esta ley, una aceleración de 2.2 a 2.5 veces con 4 núcleos equivale a que alrededor de una cuarta parte del tiempo no se aprovechó en paralelo. En esa proporción influyen también el reparto desigual de bytes entre trabajadores, el acceso al disco que comparten los cuatro trabajadores y la ejecución dentro de una máquina virtual.

2. **Los procesos y los hilos presentaron un rendimiento equivalente en este problema.** Se esperaba que la versión con hilos resultara más rápida, puesto que la versión con `fork()` debe transferir 44.5 MB de datos comprimidos por tuberías y copiarlos entre espacios de memoria. No obstante, la diferencia promedio entre ambas fue de apenas unas decenas de milisegundos, muy inferior a la variación entre corridas. Además, cada versión obtuvo el menor tiempo en corridas distintas, e incluso la versión con procesos resultó ligeramente más rápida en la descompresión (cuadros 2.2 y 2.3). Este resultado indica que el costo dominante es la compresión en sí y que, en comparación, el costo de la comunicación por tuberías es reducido. Por consiguiente, la elección entre ambos modelos depende de otros criterios. Los procesos ofrecen aislamiento ya que un error de memoria en un hijo no corrompe a los demás; los hilos, en cambio, simplifican el intercambio de datos, a cambio de exigir que todo el código compartido sea seguro entre hilos.

3. **La memoria compartida obliga a revisar todo el código, no solo**

el código nuevo. El error más difícil de detectar en este proyecto se encontraba en código común que ya había superado todas las pruebas: una variable global utilizada por `qsort()`. Dicha variable era segura en un solo hilo o en procesos separados, pero no entre hilos. Con memoria compartida, cualquier estado global constituye un posible punto de conflicto. En este sentido, la mejor sincronización es aquella que resulta innecesaria porque cada hilo trabaja sobre datos propios.

4. **La comunicación por tuberías exige prever los bloqueos.** Una implementación directa, que lee una tubería tras otra, puede quedar bloqueada, dado que la capacidad de una tubería es mucho menor que el volumen de datos que produce cada hijo. La multiplexación de la lectura con `poll()` resolvió este problema sin añadir complejidad a los hijos.

5. **Una implementación correcta no es necesariamente eficiente.** El primer decodificador producía resultados correctos y superaba todas las pruebas de integridad; sin embargo, tardaba más de tres minutos. Únicamente la medición del tiempo permitió detectar que el costo se concentraba en la búsqueda lineal realizada por cada bit. El aprovechamiento de la estructura del código canónico redujo el tiempo unas 53 veces, una mejora muy superior a la que aporta el paralelismo con 4 núcleos. Por lo tanto, la elección del algoritmo tiene mayor peso que la cantidad de procesadores.

6. **El reparto estático presenta limitaciones.** La división de los archivos en bloques contiguos es sencilla y funciona adecuadamente con este corpus. Aun así, los libros tienen tamaños muy distintos, de modo que un trabajador puede recibir más bytes que otro y terminar después. Un reparto dinámico, en el que cada trabajador toma el siguiente archivo disponible de una cola compartida, equilibraría mejor la carga; por ello, constituye una mejora natural para un trabajo futuro.

Referencias

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference* (pp. 483–485). Association for Computing Machinery. doi: 10.1145/1465482.1465560

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

Huffman, D. A. (1952). A method for the construction of minimum-redundancy codes. *Proceedings of the IRE*, *40*(9), 1098–1101. doi: 10.1109/JRPROC.1952.273898

Kerrisk, M. (2010). *The Linux programming interface: A Linux and UNIX system programming handbook*. No Starch Press.

Moffat, A., y Turpin, A. (1997). On the implementation of minimum redundancy prefix codes. *IEEE Transactions on Communications*, *45*(10), 1200–1207. doi: 10.1109/26.634683

OpenSSL Project. (s.f.). *EVP_DigestInit — OpenSSL 3.5 documentation.* `https://docs.openssl.org/3.5/man3/EVP_DigestInit/`.

Project Gutenberg. (s.f.). *Top 100 EBooks last 30 days.* `https://www.gutenberg.org/browse/scores/top#books-last30`.

Rivest, R. L. (1992). *The MD5 message-digest algorithm* (RFC n.º 1321). Internet Engineering Task Force. doi: 10.17487/RFC1321

Schwartz, E. S., y Kallick, B. (1964). Generating a canonical prefix encoding. *Communications of the ACM*, *7*(3), 166–169. doi: 10.1145/363958.363991

Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, *27*(3), 379–423. doi: 10.1002/j.1538-7305.1948.tb01338.x

Silberschatz, A., Galvin, P. B., y Gagne, G. (2018). *Operating system concepts* (10.ª ed.). Wiley.

Stix, G. (1991). Profile: David A. Huffman. Encoding the "neatness" of ones and zeroes. *Scientific American*, *265*(3), 54–58.

The GNOME Project. (s.f.). *GLib.idle_add — GLib 2.0 reference.* `https://docs.gtk.org/glib/func.idle_add.html`.

The Linux man-pages project. (s.f.-a). *clock_gettime(2) — Linux manual page.* `https://man7.org/linux/man-pages/man2/clock_gettime.2.html`.

The Linux man-pages project. (s.f.-b). *fork(2) — Linux manual page.* `https://man7.org/linux/man-pages/man2/fork.2.html`.

The Linux man-pages project. (s.f.-c). *open_memstream(3) — Linux manual page.* https://man7.org/linux/man-pages/man3/open_memstream.3.html.

The Linux man-pages project. (s.f.-d). *pipe(7) — Linux manual page.* https://man7.org/linux/man-pages/man7/pipe.7.html.

The Linux man-pages project. (s.f.-e). *poll(2) — Linux manual page.* https://man7.org/linux/man-pages/man2/poll.2.html.

The Linux man-pages project. (s.f.-f). *pthread_mutex_lock(3p) — POSIX programmer's manual.* https://man7.org/linux/man-pages/man3/pthread_mutex_lock.3p.html.

The Linux man-pages project. (s.f.-g). *pthreads(7) — Linux manual page.* https://man7.org/linux/man-pages/man7/pthreads.7.html.

The Linux man-pages project. (s.f.-h). *sysconf(3) — Linux manual page.* https://man7.org/linux/man-pages/man3/sysconf.3.html.

The Linux man-pages project. (s.f.-i). *wait(2) — Linux manual page.* https://man7.org/linux/man-pages/man2/wait.2.html.